EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 06 — January 2, 2026

Author: Jonathan Dutoit

Professor: Mathias Payer

Web and Software Bugs

Learning goals

- Web applications and regular application have a lot in common
- Understand the impact of preprocessing attacker-controlled data
- Differentiate between types of bugs and what potential attacks they enable

Web Application Security

Open Web Application Security Project (OWASP)

Top 10: Common Vulnerabilities

Goal: **Achieving arbitrary code execution**

- Cross-Site Scripting (XSS): Executing JavaScript code
- SQL Injection: Executing SQL code
- LDAP Injection: Executing LDAP queries
- Command Injection: Executing shell commands

The **Top 10** project documents the 10 most critical security risks to web apps. Updated frequently (last update was 2021) and has cross-references to CWEs (Common Weakness Enumeration).

- **A01 Broken Access Control**: Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits
  - CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
    CWE-201: Insertion of Sensitive Information Into Sent Data, and
    CWE-352: Cross-Site Request Forgery
  - Reduce attack surface by "deny by default"

  - Implement access control mechanisms once, reuse through app (user session)
  - Log access control failures
  - Rate limit API calls
  - **Real world example**: Accessing everyone's Google Drive files with Google Classrooms.

- **A02 Cryptographic Failures**: The first thing is to determine the protection needs of data in transit and at rest. For example, passwords, credit cards numbers, health records, personal information, and business secrets require extra protection.
  - CWE-259: Use of Hard-coded ***"default"*** Password,
    CWE-327: Broken or Risky Crypto Algorithm, and
    CWE-331: Insufficient Entropy
  - Reduce attack surface by storing minimal data
  - Ensure up-to-date algorithms and implementations
  - Disable legacy protcols

- **A03 Injection**: Code injection is possible when attacker controlled data is handled incorrectly and ends up being parsed as code
  - CWE-79: Cross-site Scripting,
    CWE-89: SQL Injection, and
    CWE-73: External Control of File Name or Path
  - Reduce attack surface by shifting to safe API
  - Sanitize user input through validation and escaping
  - Validate input whenever possible
  - Escape remaining characters when necessary

- **A04 Insecure Design**: Insecure design is a broad category representing different weakness, expressed as "missing or ineffective control design"
  - CWE-209: Generation of error message containing sensitive information,
    CWE-256: Unprotected Storage of Credentials,
    CWE-501: Trust Boundary Violation, and
    CWE-522: Insufficiently Protected Credentials
  - Prevent by establishing secure development lifecycle
  - Use threat modeling where feasible
  - Segregate and compartmentalize where possible

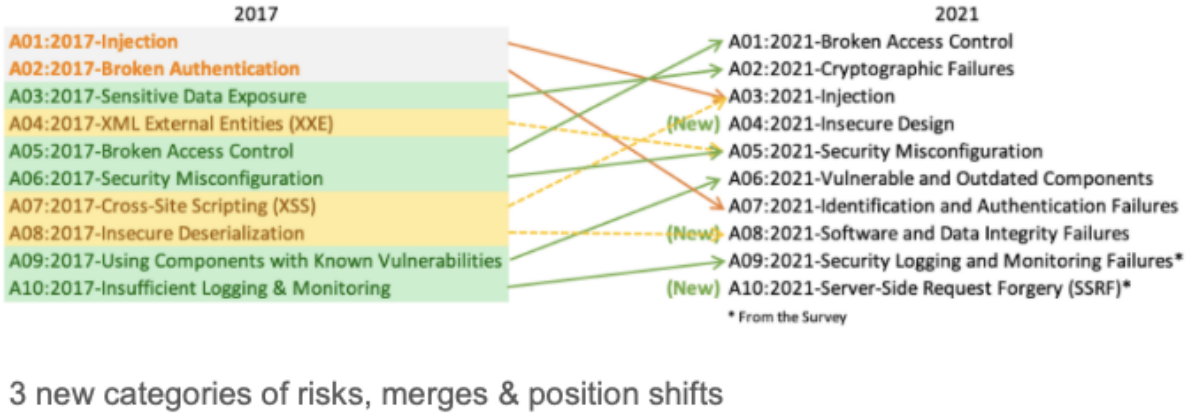
- **A05 Security Misconfiguration**: Software is getting more configurable and administrators need to carefully balance features and security. Any software along a complex stack can be misconfigured.
  - CWE-16: Configuration and
    CWE-611: Impostor Restriction of XML External Entity Reference
  - Document the software stack and its configuration

  - Harden all configuration
  - Minimize exposed features

- **A06 Vulnerable / Outdated Components**: Software may have known vulnerabilities that can be abused by an attacker.
  - CWE-1104: Use of Unmaintained Third-Party Components
  - Minimize attack surface by removing or disabling unused components
  - Track inventory of software and follow upstream security notices
  - Monitor for available patches, if patching is not possible mitigate through firewall or high-level interception
- **A07 Identification / Authentication Failures**: Confirmation of the user's identity, authentication, and session management is critical to protect against authentication-related attacks
  - CWE-297: Improper Validating of Certificate with Host Mismatch,
    CWE-287: Improper Authentication, and
    CWE-384: Session Fixation
  - Reduce attack surface through 2FA
  - Check for good password hygiene (length, weak passwords, policies)
  - Limit or increasingly delay failed login attempts
- **A08 Software and Data Integrity Failures**: Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations. An example of this is where an application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs)
  - CWE-829: Inclusion of Functionality from Untrusted Control Sphere,
    CWE-494: Download of Code Without Integrity Check, and
    CWE-502: Deserialization of Untrusted Data
  - Mitigate by verifying (through signatures) that data comes from verified source and has not been tampered
  - Review code and configuration changes to limit potential mistakes
- **A09 Security and Logging Monitoring**: Breaches will happen. Document ongoing security activities and safely log actions to enable detection and analysis of breaches.
  - CWE-778: Insufficient Logging,
    CWE-117: Improper Output Neutralization for Logs,
    CWE-223: Omission of Security-relevant information, and
    CWE-532: Insertion of Sensitive Information into Log File
  - Mitigate by logging failed logins, authentication, or access control checks
  - Ensure high-value transactions have an audit trail
  - Protect log data through integrity or other safeguards

- **A10 Server Side Request Forgery**: SSRF flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).
  - CWE-918: Server-Side Request Forgery (SSRF)
  - Added due to community vote (not observed attack data)
  - Prevent by implementing defense in depth
  - Deny by default, sanitize and validate all user data based on allow-list

*OWASP Top 10 Follow Trends



3 new categories of risks, merges & position shifts

Arbitrary Code Execution

Code injection is possible when user-supplied data escapes control. The vulnerability comes from special characters in **user inputs** can trigger an action in the context (HTML, SQL, LDAP, OS commands, XML, XSL, SMTP).

The impact is an attacker-controlled data becomes code that may be abused to escape the intended behavior of the original code.

Recent example: Log4j Exploit used in many java-based applications.

Protection against Injection

Deny by default, otherwise inspect received data **twice**

When receiving: Input validation When using: Encode data

- Escape (encode) special characters when you use them

Separate code and data (e.g., SQL prepared statements)

Web frameworks offer sanitization functions!

For every request, check if the requesting user has adequate permissions!

Software Security

threats

- Stack-based overflows
- Direct overwrites through pointer manipulation
- Heap-based overflows and overwrites
- Underflows instead of overflows
- Exception-based hijacking

Note: In this lecture we focus more on stack-based/buffer overflows (variable overwrite, overflowing return address, code injection, etc.). More information about software security is given by CS-412 in Spring.

Programmers are not perfect, all software therefore has bugs. Some bugs make the software vulnerable and sooner or later adversaries will find a way to exploit those vulnerabilities. We make a difference between **logic bugs** (e.g., forgetting to check access permissions) and **bugs that violate memory and type safety**, leading to memory corruption.

Defense Strategies

1. Memory safety: stop memory corruption by using safe languages (Java/Rust)
2. Integrity: enforce integrity of reads/writes by doing Write Integrity Testing, Code-Pointer integrity, etc...
3. Randomization: Probabilistic defenses by randomizing locations, code, data, or pointer values.
   - Oscar made a really good point about Anti-Cheating in games often randomized variable locations so that cheats can't work
4. Flow Integrity: protect control transfers by verifying data-flow and control-flow integrity.
5. If we cannot assure all steps above, the our program is susceptible to Attack: Control-flow hijack

Defense: DEP Hardware Support

Page table extension, introduce NX-bit (No eXecute bit) which was conceived in the early 2000s following the advancement of operating systems supporting 64-bit architecture instead of only 32 bits. Having a 64-bit architecture allowed space for the DEP.

This is an additional bit for every mapped virtual page. If the bit is set, then data on that page cannot be interpreted as code and the processor will trap if control flow reaches that page.

Defense: Address Space Layout Randomization (ASLR)

ASLR is a **pratical form** of Address Space Randomization. It focuses on blocks of memory (i.e., it is coarse grained).

Heap, stack, code, executable, mmap regions end up at random addresses.

ASLR is inherently page-based, limiting cost of shuffling and it initializes when process starts (never re-randomized). It can also be combined with DEP for better security.

Defense: Stack Canaries

Early attacks overflowed stack-based buffers to inject code. Memory safety would mitigate this problem but adding full safety checks is infeasible due to high performance overhead.

Instead of checking each pointer dereference, check integrity only before important data is accessed, e.g., before accessing a code pointer.

Assumption: we only prevent RIP (return instruction pointer) control-flow hijack attacks.

Therefore, only need to protect the integrity of the RIP. Key insight: buffer overflows are only possible after pointer arithmetic.

Hence, the strategy is to place a canary after a potentially vulnerable buffer and after check the integrity of the canary before the function returns.

Limitation: the stack canary only protects against ***continuous overwrites*** if (and only if) the attacked does not know the canary. An alternative is to encrypt the return instruction pointer by xoring it with a secret

Hence, Canary is stored in a safe location, copied to stack slot in prologue and checked in epilogue before returning. It protects against most stacked-based hijack attacks and it's simple, low-overhead compiler-based defense.